AWS システム設計資料 - スケーラブルなRESTful APIシステム

システム概要

サーバーレスアーキテクチャを活用し、自動スケーリング対応の REST API を構築するシステム。API Gateway + Lambda により、インフラ管理なしに高性能で堅牢なバックエンドを実現します。

対象用途:

- モバイルアプリケーションバックエンド
- ウェブアプリケーション API
- マイクロサービス
- IoT デバイス連携
- データ取得・操作インターフェース

アーキテクチャ構成

```
クライアント (モバイル/Web)
    ↓
  HTTPS
    ↓
Route 53 (DNS)
    ↓
API Gateway
├─ GET /users
├─ POST /users
├─ GET /users/{id}
├─ PUT /users/{id}
└─ DELETE /users/{id}
    ↓ Lambda 統合
  Lambda Function
    ├─ リクエスト処理
    ├─ ビジネスロジック
    └─ レスポンス生成
    ↓
データベース層
├─ DynamoDB (NoSQL)
├─ RDS MySQL (RDBMS)
└─ ElastiCache (キャッシュ)
    ↓
  CloudWatch (ログ・監視)
```

使用するAWSサービス

サービス	役割	責務内容

サービス	役割	責務内容
API Gateway	API エンドポイント	リクエスト処理、認証、レート制限
Lambda	コンピュート	ビジネスロジック実行
DynamoDB	NoSQL DB	高速キー・バリューストレージ
RDS	リレーショナル DB	トランザクション処理
ElastiCache	キャッシュ層	高速データ取得
CloudWatch	監視・ログ	メトリクス、ログ、アラーム
Cognito	認証・認可	ユーザー認証、JWT 生成

詳細設定値

API Gateway 設定

```
API 名: user-management-api
プロトコル: REST

エンドポイント設定:
  - エンドポイントタイプ: Regional
  - DNS: api.example.com

ステージ設定:
  - ステージ名: prod, dev, staging
  - キャッシュクラスター: 0.5 (小)
  - キャッシュ TTL: 300秒 ( 5分 )

スロットル設定:
  - リクエストレート: 10,000 req/s
  - バーストレート: 20,000 req/s

リソース定義:

1. GET /users
   └ Lambda 統合
      - Lambda: getUsersList
      - キャッシュ: 有効 (TTL 300s)
      - リクエスト検証: なし

2. POST /users
   └ Lambda 統合
      - Lambda: createUser
      - キャッシュ: 無効
      - リクエスト検証: JSON スキーマ有効

3. GET /users/{userId}
   └ Lambda 統合
      - Lambda: getUser
      - パスパラメータ: userId (文字列)
```

```
      - キャッシュ: 有効 (TTL 600s)

4. PUT /users/{userId}
   └ Lambda 統合
      - Lambda: updateUser
      - リクエスト検証: JSON スキーマ

5. DELETE /users/{userId}
   └ Lambda 統合
      - Lambda: deleteUser

認証設定:
  - タイプ: AWS_IAM + Cognito
  - Cognito ユーザープール: user-pool-prod
  - 認可スコープ: api/read, api/write

CORS 設定:
  - 許可オリジン: https://example.com, https://app.example.com
  - 許可メソッド: GET, POST, PUT, DELETE, OPTIONS
  - 許可ヘッダー: Content-Type, Authorization, X-API-Key
  - 公開ヘッダー: x-request-id, x-amzn-requestid
  - 認証情報: 有効
```

Lambda 関数設定

```
関数名: user-management-prod

実行時: Python 3.11

基本設定:
  - メモリ: 512 MB
  - タイムアウト: 30秒
  - 一時ストレージ (/tmp): 10,240 MB
  - アーキテクチャ: x86_64

環境変数:
  - DB_HOST: rds-instance.xxxx.rds.amazonaws.com
  - DB_NAME: user_database
  - DB_USER: admin
  - DYNAMODB_TABLE: Users
  - CACHE_ENDPOINT: cache.xxxx.cache.amazonaws.com:6379
  - LOG_LEVEL: INFO
  - API_TIMEOUT: 25

VPC 設定:
  - VPC: vpc-xxxxx
  - セキュリティグループ: sg-lambda-db
  - サブネット: subnet-private-1a, subnet-private-1b
  - NAT ゲートウェイ: 経由でアウトバウンド

IAM 実行ロール: lambda-execution-role
```

```
  - アクセス許可:
    * logs:CreateLogGroup
    * logs:CreateLogStream
    * logs:PutLogEvents
    * dynamodb:GetItem
    * dynamodb:PutItem
    * dynamodb:UpdateItem
    * dynamodb:DeleteItem
    * dynamodb:Query
    * rds-db:connect
    * ec2:CreateNetworkInterface
    * ec2:DescribeNetworkInterfaces
    * ec2:DeleteNetworkInterface

レイヤー設定:
  - 名前: shared-dependencies
  - 内容: requests, boto3, mysql-connector-python
  - バージョン: 1
```

Lambda 関数コード例 (Python)

```python
import json
import boto3
import logging
from datetime import datetime
import os

logger = logging.getLogger()
logger.setLevel(os.environ.get('LOG_LEVEL', 'INFO'))

dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table(os.environ['DYNAMODB_TABLE'])

def lambda_handler(event, context):
    """
    メインハンドラー
    """
    try:
        http_method = event['httpMethod']
        path = event['path']

        if http_method == 'GET' and path == '/users':
            return get_users_list(event)
        elif http_method == 'POST' and path == '/users':
            return create_user(event)
        elif http_method == 'GET' and '/users/' in path:
            user_id = event['pathParameters']['userId']
            return get_user(user_id)
        elif http_method == 'PUT' and '/users/' in path:
            user_id = event['pathParameters']['userId']
            return update_user(user_id, event)
```

```
        elif http_method == 'DELETE' and '/users/' in path:
            user_id = event['pathParameters']['userId']
            return delete_user(user_id)
        else:
            return error_response(404, 'Not Found')

    except Exception as e:
        logger.error(f'Unexpected error: {str(e)}')
        return error_response(500, 'Internal Server Error')

def get_users_list(event):
    """
    ユーザー一覧取得
    """
    try:
        query_params = event.get('queryStringParameters') or {}
        limit = int(query_params.get('limit', 20))
        limit = min(limit, 100)  # 最大100件

        response = table.scan(Limit=limit)

        return success_response(200, {
            'users': response.get('Items', []),
            'count': response.get('Count', 0),
            'scanned_count': response.get('ScannedCount', 0)
        })

    except Exception as e:
        logger.error(f'Error in get_users_list: {str(e)}')
        return error_response(500, 'Failed to retrieve users')

def create_user(event):
    """
    ユーザー作成
    """
    try:
        body = json.loads(event.get('body', '{}'))

        # バリデーション
        required_fields = ['email', 'name']
        if not all(field in body for field in required_fields):
            return error_response(400, 'Missing required fields')

        user_id = generate_user_id()
        user_data = {
            'userId': user_id,
            'email': body['email'],
            'name': body['name'],
            'createdAt': datetime.utcnow().isoformat(),
            'updatedAt': datetime.utcnow().isoformat()
        }

        table.put_item(Item=user_data)
```

```
        return success_response(201, user_data)

    except json.JSONDecodeError:
        return error_response(400, 'Invalid JSON')
    except Exception as e:
        logger.error(f'Error in create_user: {str(e)}')
        return error_response(500, 'Failed to create user')

def get_user(user_id):
    """
    ユーザー詳細取得
    """
    try:
        response = table.get_item(Key={'userId': user_id})

        if 'Item' not in response:
            return error_response(404, 'User not found')

        return success_response(200, response['Item'])

    except Exception as e:
        logger.error(f'Error in get_user: {str(e)}')
        return error_response(500, 'Failed to retrieve user')

def update_user(user_id, event):
    """
    ユーザー更新
    """
    try:
        body = json.loads(event.get('body', '{}'))

        # 更新可能なフィールド
        update_fields = ['name', 'email']
        update_expr_parts = []
        expr_attr_values = {}

        for field in update_fields:
            if field in body:
                update_expr_parts.append(f'{field} = :{field}')
                expr_attr_values[f':{field}'] = body[field]

        if not update_expr_parts:
            return error_response(400, 'No fields to update')

        update_expr_parts.append('updatedAt = :updated_at')
        expr_attr_values[':updated_at'] = datetime.utcnow().isoformat()

        update_expression = 'SET ' + ', '.join(update_expr_parts)

        response = table.update_item(
            Key={'userId': user_id},
            UpdateExpression=update_expression,
            ExpressionAttributeValues=expr_attr_values,
            ReturnValues='ALL_NEW'
```

```
        )

        return success_response(200, response['Attributes'])

    except json.JSONDecodeError:
        return error_response(400, 'Invalid JSON')
    except Exception as e:
        logger.error(f'Error in update_user: {str(e)}')
        return error_response(500, 'Failed to update user')

def delete_user(user_id):
    """
    ユーザー削除
    """
    try:
        table.delete_item(Key={'userId': user_id})
        return success_response(204, None)

    except Exception as e:
        logger.error(f'Error in delete_user: {str(e)}')
        return error_response(500, 'Failed to delete user')

def success_response(status_code, body):
    """
    成功レスポンス
    """
    return {
        'statusCode': status_code,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*',
            'X-Request-Id': context.request_id
        },
        'body': json.dumps(body)
    }

def error_response(status_code, message):
    """
    エラーレスポンス
    """
    return {
        'statusCode': status_code,
        'headers': {
            'Content-Type': 'application/json',
            'Access-Control-Allow-Origin': '*'
        },
        'body': json.dumps({'error': message})
    }

def generate_user_id():
    """
    ユーザーID生成
    """
```

```
    import uuid
    return str(uuid.uuid4())
```

DynamoDB テーブル設定

```
テーブル名: Users

キャパシティモード: オンデマンド（自動スケーリング）
  - 読み込み: 従量課金
  - 書き込み: 従量課金
  - 最大スループット: 40,000 WCU, 40,000 RCU

パーティションキー:
  - 属性名: userId
  - 型: 文字列 (S)

ソートキー: なし

TTL 設定:
  - 属性名: expirationTime
  - 有効: ○
  - 90日後の自動削除

グローバルセカンダリインデックス (GSI):
  1. インデックス名: emailIndex
     - パーティションキー: email
     - 型: 文字列
     - キャパシティ: オンデマンド
     - 投影タイプ: ALL

  2. インデックス名: createdAtIndex
     - パーティションキー: createdAt
     - ソートキー: userId
     - キャパシティ: オンデマンド
     - 投影タイプ: KEYS_ONLY

ストリーム仕様:
  - ストリームスペック: 有効
  - ビューイメージ: NEW_AND_OLD_IMAGES

ポイントインタイムリカバリ (PITR):
  - 有効: ○
  - 保持期間: 35日

タグ付け:
  - Environment: production
  - CostCenter: backend
  - Owner: platform-team
```

RDS MySQL インスタンス設定

```
DB インスタンス識別子: user-database-prod

エンジン設定:
  - DB エンジン: MySQL
  - バージョン: 8.0.35
  - インスタンスクラス: db.t3.micro

ストレージ:
  - ストレージタイプ: gp3
  - 割り当てストレージ: 100 GB
  - IOPS: 3000
  - スループット: 125 MB/s

可用性:
  - マルチAZ: 有効
  - 同期レプリケーション
  - フェイルオーバー時間: 数秒

バックアップ:
  - 自動バックアップ: 有効
  - バックアップ保持期間: 30日
  - バックアップウィンドウ: 03:00-04:00 UTC
  - コピー自動バックアップ: 有効（別リージョン）

ネットワーク:
  - VPC: vpc-xxxxx
  - セキュリティグループ: sg-rds-prod
  - パブリックアクセス: 無効
  - VPC セキュリティグループルール:
    * インバウンド: Lambda SG からのみ 3306 許可

パラメータグループ:
  - max_connections: 100
  - slow_query_log: 1
  - long_query_time: 2
  - character_set_server: utf8mb4
  - collation_server: utf8mb4_unicode_ci

監視:
  - CloudWatch: 有効
  - Enhanced Monitoring: 有効 (granularity: 60s)
  - ログエクスポート: Error, General, Slow query
```

Cognito ユーザープール設定

```
ユーザープール名: user-pool-prod

ユーザー属性:
  - email（必須、一意）
  - name（必須）
```

```
    - phone_number（オプション）
    - custom:department（カスタム属性）

  パスワードポリシー:
    - 最小長: 12文字
    - 大文字: 必須
    - 小文字: 必須
    - 数字: 必須
    - 特殊文字: 必須

  MFA (多要素認証):
    - TOTP: 有効
    - SMS: 有効

  アプリクライアント設定:
    - クライアント名: api-client
    - 認証フロー: ALLOW_USER_PASSWORD_AUTH, ALLOW_REFRESH_TOKEN_AUTH
    - トークン有効期限:
      * Access Token: 1時間
      * Refresh Token: 30日
      * ID Token: 1時間

  リソースサーバー:
    - ID: api
    - スコープ:
      * read: データ読み取り
      * write: データ書き込み
      * delete: データ削除

  ホストされたUI:
    - 有効: ○
    - コールバック URL: https://example.com/callback
```

推定月額コスト

シナリオ: 月間リクエスト 1000万、月間スキャン 500万

項目	使用量	単価	月額費用
API Gateway リクエスト	1000万	$3.50/100万	$35.00
Lambda 呼び出し	1000万	$0.20/100万	$2.00
Lambda 実行時間	50,000,000 GB-秒	$0.0000166667/GB-秒	$833.33
DynamoDB 読み込み	500万 RCU	$1.25/100万 RCU	$6.25
DynamoDB 書き込み	100万 WCU	$6.25/100万 WCU	$6.25
DynamoDB ストレージ	50 GB	$1.25/GB	$62.50
RDS (db.t3.micro)	730時間	$0.017/時間	$12.41
RDS ストレージ	100 GB	$0.23/GB	$23.00

項目	使用量	単価	月額費用
CloudWatch ログ	100 GB	$0.50/GB	$50.00
合計			**$1,030.74**

セキュリティ設定

API Gateway リソースポリシー

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOnly",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*",
      "Condition": {
        "StringEquals": {
          "aws:SourceVpc": "vpc-xxxxx"
        }
      }
    },
    {
      "Sid": "DenyHTTP",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "execute-api:Invoke",
      "Resource": "execute-api:/*",
      "Condition": {
        "StringEquals": {
          "aws:PrincipalType": "Anonymous"
        }
      }
    }
  ]
}
```

Lambda IAM 実行ロールポリシー

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DynamoDBAccess",
      "Effect": "Allow",
      "Action": [
        "dynamodb:GetItem",
```

```
          "dynamodb:PutItem",
          "dynamodb:UpdateItem",
          "dynamodb:DeleteItem",
          "dynamodb:Query",
          "dynamodb:Scan"
        ],
        "Resource": "arn:aws:dynamodb:*:ACCOUNT-ID:table/Users",
        "Condition": {
          "StringEquals": {
            "aws:RequestedRegion": "ap-northeast-1"
          }
        }
      },
      {
        "Sid": "CloudWatchLogs",
        "Effect": "Allow",
        "Action": [
          "logs:CreateLogGroup",
          "logs:CreateLogStream",
          "logs:PutLogEvents"
        ],
        "Resource": "arn:aws:logs:*:ACCOUNT-ID:*"
      }
    ]
  }
```

デプロイメント手順

1. Lambda 関数パッケージング

```
# 依存関係のインストール
pip install -r requirements.txt -t ./package

# コードをパッケージに追加
cp lambda_function.py package/

# ZIP 作成
cd package
zip -r ../lambda_function.zip .
cd ..

# AWS アップロード
aws lambda create-function \
  --function-name user-management-prod \
  --runtime python3.11 \
  --role arn:aws:iam::ACCOUNT-ID:role/lambda-execution-role \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://lambda_function.zip \
  --timeout 30 \
  --memory-size 512 \
  --vpc-config SubnetIds=subnet-xxxxx,SecurityGroupIds=sg-xxxxx
```

2. API Gateway デプロイ

```
# API 作成
API_ID=$(aws apigateway create-rest-api \
  --name user-management-api \
  --description "User Management REST API" \
  --query 'id' \
  --output text)

# ステージデプロイ
aws apigateway create-deployment \
  --rest-api-id $API_ID \
  --stage-name prod \
  --stage-description "Production stage"
```

3. DynamoDB テーブル作成

```
aws dynamodb create-table \
  --table-name Users \
  --attribute-definitions \
    AttributeName=userId,AttributeType=S \
    AttributeName=email,AttributeType=S \
  --key-schema \
    AttributeName=userId,KeyType=HASH \
  --billing-mode PAY_PER_REQUEST \
  --global-secondary-indexes \
    '[{
      "IndexName": "emailIndex",
      "KeySchema": [
        {"AttributeName": "email", "KeyType": "HASH"}
      ],
      "Projection": {"ProjectionType": "ALL"}
    }]' \
  --ttl-specification AttributeName=expirationTime,Enabled=true \
  --stream-specification StreamViewType=NEW_AND_OLD_IMAGES
```

パフォーマンス最適化

1. キャッシング戦略

```
API Gateway キャッシュ:
  - GET リクエスト: TTL 300秒
  - キャッシュキー: リソースID + パラメータ

Lambda 層のキャッシング:
  - DB コネクションプーリング
```

```
  - 接続再利用

DynamoDB キャッシング:
  - DAX (DynamoDB Accelerator) 使用
  - TTL: 5分
```

2. Lambda 最適化

```
- コールドスタート削減:
  * 初期化コードを共有ライブラリに移動
  * Provisioned Concurrency 使用
  * Lambda@Edge 併用

- メモリ最適化:
  * 512 MB でベストバランス
  * オブジェクトサイズ監視
```

3. データベース最適化

```
- インデックス活用:
  * プライマリキー設計最適化
  * GSI 活用（適切に設計）

- クエリ最適化:
  * 必要カラムのみ SELECT
  * LIMIT 使用
```

監視とロギング

CloudWatch メトリクス

```
監視対象:
- API Gateway:
  * Count: リクエスト総数
  * Latency: 応答時間（平均、p99）
  * 4XXError: クライアントエラー
  * 5XXError: サーバーエラー
  * CacheHitCount: キャッシュヒット

- Lambda:
  * Invocations: 実行回数
  * Duration: 実行時間
  * Errors: エラー数
  * Throttles: スロットル
  * ConcurrentExecutions: 同時実行数
```

```
- DynamoDB:
  * ConsumedReadCapacityUnits: 読み込み容量
  * ConsumedWriteCapacityUnits: 書き込み容量
  * UserErrors: ユーザーエラー

アラーム設定:
  - 5xxError > 5% → 通知
  - Lambda Duration > 20秒 → 警告
  - DynamoDB Throttles > 0 → 通知
```